

MUON CAN OUTPERFORM DEDICATED CONTINUAL LEARNING METHODS

Sebastian George Sincari*

Faculty of Mathematics and Computer Science
University of Bucharest
Bucharest, Romania
sebastian-george.sincari@s.unibuc.ro

Bogdan Alexandru Gheorghe*

Faculty of Mathematics and Computer Science
University of Bucharest
Bucharest, Romania
bogdan-alexandru.gheorghe@s.unibuc.ro

Antonio Barbalau

Bitdefender
Bucharest, Romania
abarbalau@bitdefender.com

ABSTRACT

Parameter-efficient fine-tuning methods for continually adapting large models typically mitigate forgetting by constraining each new Low-Rank Adapter (LoRA) to interfere as little as possible with the previously learned ones. Existing state-of-the-art continual-learning (CL) methods often enforce this by penalizing the geometric overlap between new updates and past weights, which inherently causes the resulting parameter updates to become smaller and better conditioned. This raises the natural question: can interference between updates be attenuated through update geometry alone, without any task-aware directionality? We probe this by changing the optimizer instead of the method. We train a plain LoRA with Muon, an optimizer that orthogonalizes and normalizes each update by construction, and compare it against established recent CL methods under a matched-learning-rate, early-stopped protocol. Surprisingly, plain LoRA+Muon—using *no* continual-learning-specific regularization—matches or outperforms these dedicated methods on both benchmarks. We report these preliminary results as evidence that a meaningful part of the advantage usually attributed to dedicated CL methods may instead be explained by the geometry of the optimizer’s updates, and we outline the experiments needed to confirm the effect.

1 INTRODUCTION

Continually fine-tuning a pretrained model on a sequence of tasks remains difficult because of *catastrophic forgetting*: adapting to a new task tends to overwrite the representations learned for previous ones. In the large-model regime, full fine-tuning is rarely practical, so most recent work builds on parameter-efficient fine-tuning, and on LoRA in particular (Hu et al., 2022). A family of continual-learning methods then augments LoRA with mechanisms that explicitly protect previously acquired knowledge.

Two representative instances are O-LoRA (Wang et al., 2023a), which forces the low-rank update of a new task to be orthogonal to the subspaces used by earlier tasks, and ELLA (Das Biswas et al., 2026), which adds a regularization penalty that keeps a new adapter from overwriting the accumulated past weights. Although their formulations differ, both share a single underlying behavior: by restricting the directions a new update may take, they also *shrink the effective size of that update*. This observation motivates our central question. If the operative ingredient is a smaller, well-conditioned update, is a continual-learning-specific mechanism actually required to obtain it, or can the same effect be produced at the level of the optimizer?

To test this, we keep the model plain—a standard LoRA with no CL-specific penalty—and instead change how the update is computed, using Muon (Jordan et al., 2024), an optimizer that orthogonalizes and normalizes each update by construction. Under a controlled, fair comparison protocol, we find that plain LoRA+Muon matches or exceeds ELLA and O-LoRA. We deliberately frame this as a surprising preliminary finding rather than a finished result: our goal here is to document an effect we did not expect and to motivate the follow-up study, not to claim a general solution to continual learning.

* Equal contribution.

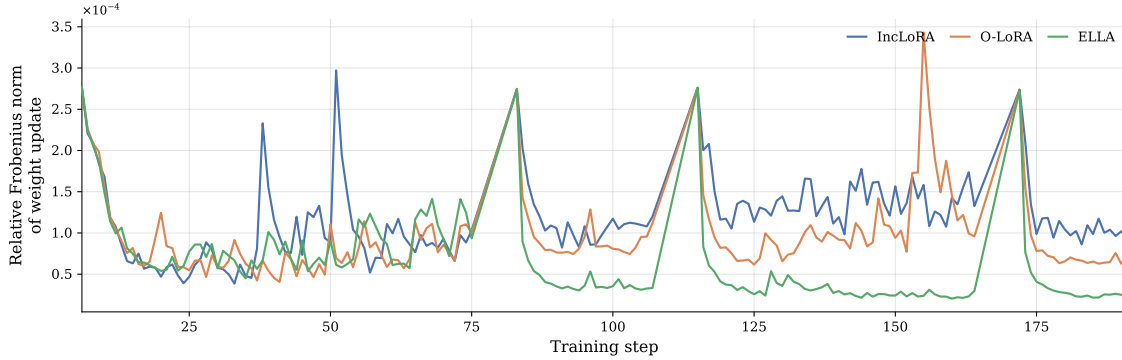


Figure 1: Evolution of the global relative Frobenius norm of weight updates during training (Appendix A). The visualization highlights the differences in learning dynamics, plasticity, and stability when employing IncLoRA, O-LoRA and ELLA trained with AdamW on Standard CL Benchmark over Order 1.

Contributions. This work-in-progress makes two preliminary contributions:

- We test whether the mitigation of catastrophic forgetting can be obtained from the optimizer alone, by training IncLoRA with Muon under a matched-learning-rate, early-stopped protocol (Section 3).
- We report that IncLoRA+Muon, without any CL-specific regularization, is competitive with or better than ELLA on our benchmarks (Section 4), and discuss what this does and does not yet show (Section 5).

2 BACKGROUND

Continual learning with LoRA. We consider the standard setting in which a pretrained model is adapted to a sequence of tasks $T_1, \dots, T_K$ seen one at a time. For each task a LoRA adapter is trained and then merged into the frozen backbone before moving to the next task. We measure performance with the metrics common in this literature: overall accuracy across tasks after the full sequence (OA, higher is better), backward transfer (BWT, where values closer to zero indicate less forgetting), and forward transfer (FWT). Throughout, our reference benchmarks are the Standard CL Benchmark (Zhang et al., 2015) and TRACE (Wang et al., 2023b), a more challenging benchmark that provides a longer and more diverse sequence of tasks.

Constraint-based methods produce smaller updates. O-LoRA (Wang et al., 2023a) constrains the low-rank update of each new task to be orthogonal to the subspaces spanned by previous tasks, so that learning a new task overwrites the directions used by old tasks as little as possible. ELLA (Das Biswas et al., 2026) instead adds a regularization term that penalizes the overlap between the new adapter and the accumulated past weights. In both cases, restricting the admissible update directions has a measurable consequence: the norm of the parameter update is reduced relative to a naive incremental LoRA baseline (IncLoRA), which simply stacks adapters with no constraint. Figure 1 illustrates this effect for the constraint-based methods.

Muon. Muon (Jordan et al., 2024) is an optimizer for the weight matrices of a network that replaces each update of the raw gradient by an (approximately) orthogonal one, obtained through a few Newton–Schulz iterations. Intuitively, the resulting update has its singular values normalized toward one, which means Muon directly controls the geometry and the scale of each step *by construction*, rather than through an auxiliary loss term. This is precisely the property we want to isolate: a smaller, well-conditioned update obtained at the optimizer level, without any continual-learning-specific machinery.

3 METHODS

Hypothesis. Standard CL methods mitigate forgetting by explicitly constraining updates to minimize interference with past tasks. We hypothesize that the geometry of the update is the primary driver of this success. Using the Muon optimizer, which enforces orthogonal updates, we posit that models might naturally minimize feature overlap by distributing updates across orthogonal singular dimensions. Furthermore, Chizat et al. (2019) describe a lazy training regime, where a network barely moves from its initialization and acts like a linear model, learning with a fixed

kernel instead of adapting its features. We do not claim that AdamW (Loshchilov & Hutter, 2019) actually causes this regime; we only use it as a way to interpret what we observe. AdamW and Muon shape the update differently: AdamW rescales each coordinate on its own, while Muon acts on the whole gradient matrix at once (Lau et al., 2025), and Jordan et al. (2024) show that Muon makes all singular directions equally strong. Our secondary hypothesis follows from this difference: AdamW tends to put most of each update into a few dominant directions, while Muon spreads it evenly across all orthogonal directions, which we suspect pushes the network out of the lazy regime toward richer feature learning in the CL setting.

Methods compared. We compare three continual-learning methods, IncLoRA, O-LoRA, and ELLA, each trained with both AdamW and Muon, yielding six method-optimizer combinations in total. We evaluate every combination in two distinct settings: (i) the Standard CL Benchmark (across task Orders 1, 2, and 3 from the ELLA protocol), where the tasks are relatively well-aligned text classification benchmarks; and (ii) the TRACE Benchmark (corresponding to Order 7 in the ELLA protocol). Unlike the Standard Benchmark, TRACE features a highly diverse sequence of tasks spanning code, multilingual data, and reasoning.¹ This higher distribution shift and inherent cross-task orthogonality allow us to evaluate how both optimizers and CL constraints behave under severe domain changes.

Models and benchmarks. The two settings differ in model, benchmark, and training protocol.

(i) ELLA protocol. Following Das Biswas et al. (2026), we use the T5 model (Raffel et al., 2020) on the Standard CL Benchmark (Orders 1–3), keeping the same hyperparameters as the original paper. Since the ELLA paper specifies LoRA’s rank at a value of 8 but does not report the scaling factor value α , we swept three values (16, 32, 64) and we selected 16, as it most closely recovered the reported baseline.

(ii) Isolating the geometric effect of Muon. Here we use the Flan-T5 model (Chung et al., 2024) on the TRACE Benchmark. Our goal is to isolate the purely geometric effect of Muon’s update from any difference in step size; we therefore equalize the update magnitude across optimizers and replace the fixed epoch budget with early stopping, letting each model train until the loss stagnates.

The equalization of the effective update magnitude is motivated by two observations. First, empirically: at the AdamW anchor learning rate, Muon was visibly undertrained, settling at a noticeably higher training loss than the matched AdamW runs, which indicates that its effective step size was too small. Second, theoretically: Liu et al. (2025) derive a per-parameter update-scale rule that matches Muon’s effective update magnitude to that of AdamW, which we adapt to our setting. Concretely, for an adapter update of shape $m \times n$ we set

$$\eta_{\text{Muon}} = \eta_{\text{AdamW}} \cdot \sqrt{\max(m, n)},$$

keeping AdamW as the anchor and scaling Muon accordingly. We give the derivation of why this choice equalizes the two update magnitudes in Appendix B.

Across all methods we control for confounding factors: they share the same data, the same task order, the same LoRA rank and scaling, and the same early-stopping criterion. As in the original LoRA paper (Hu et al., 2022), LoRA adapters are attached to the query and value heads. This ensures that no method is advantaged by a longer or shorter training budget.

4 RESULTS

Metrics. We report forward transfer (FWT), backward transfer (BWT), and overall accuracy (OA, the average end-of-sequence accuracy introduced in Section 2); recall that for BWT values closer to zero indicate less forgetting. Table 1 reports the TRACE comparison and Table 2 the Standard CL comparison; in both, every method is trained once with AdamW and once with Muon. As noted in Section 3, these are single runs that we read qualitatively; multi-seed results are in progress (Section 6).

Our pipeline reproduces ELLA, and the unconstrained baseline is already strong. On Standard CL Benchmark with AdamW (Table 2), our ELLA run reaches an OA of 79.15, matching the 79.9 reported by Das Biswas et al. (2026) to within single-seed variation and confirming that our pipeline behaves as intended. On the same setting, our IncLoRA baseline, which stacks adapters with no continual-learning constraint, reaches 77.70, far above the 63.6 reported for IncLoRA in prior work.

¹The eight TRACE tasks are C-STANCE (Zhao et al., 2023), FOMC (Shah et al., 2023), MeetingBank (Hu et al., 2023), Py150 (Lu et al., 2021), ScienceQA (Lu et al., 2022), NumGLUE-cm and NumGLUE-ds (Mishra et al., 2022), and 20Minuten (Rios et al., 2021).

Table 1: TRACE Benchmark (Order 7). Forward transfer (FWT), backward transfer (BWT, closer to zero is better), and overall accuracy (OA). Best OA per optimizer in bold. Single runs.

	AdamW			Muon		
	FWT	BWT	OA	FWT	BWT	OA
IncLoRA	-2.18	-0.42	25.14	-5.72	-0.36	32.06
O-LoRA	-2.26	-0.72	24.30	-6.09	-0.10	31.22
ELLA	-1.95	-0.62	24.81	-3.90	-0.07	28.60

Table 2: Standard CL Benchmark. Per-order accuracy and overall accuracy (OA, average over Orders 1–3). Best OA per optimizer in bold. Single runs.

	AdamW				Muon			
	Order 1	Order 2	Order 3	OA	Order 1	Order 2	Order 3	OA
IncLoRA	79.52	71.92	81.66	77.70	84.43	84.05	79.52	82.66
O-LoRA	80.03	82.16	80.87	81.02	82.08	83.74	75.62	80.48
ELLA	80.30	80.52	76.65	79.15	71.14	70.50	73.59	71.74

With Muon, the unconstrained baseline becomes the strongest method. The central result is visible in both tables: IncLoRA+Muon attains the highest OA of all six method-optimizer combinations on each benchmark, 32.06 on TRACE and 82.66 on Standard CL, exceeding ELLA under either optimizer (ELLA reaches 24.81/28.60 on TRACE and 79.15/71.74 on Standard CL with AdamW/Muon). An adapter with no CL-specific regularization, trained with an optimizer that controls update geometry by construction, thus outperforms the dedicated CL method it is compared against.

Optimizer geometry and explicit constraints are substitutes, not complements. Muon’s benefit is concentrated on the unconstrained method, while on constrained methods it becomes redundant or even conflicting. On Standard CL, switching from AdamW to Muon improves IncLoRA by +4.96 OA, leaves O-LoRA essentially unchanged (-0.54), and degrades ELLA by -7.41 . On TRACE the relative ordering is the same: Muon improves both IncLoRA and O-LoRA by 6.92, and ELLA only by 3.79, so that under Muon ELLA is the weakest of the three. This is consistent with the mechanism of Figure 1: O-LoRA and ELLA already shrink each update through their constraints, and Muon shrinks it through orthogonalization, so applying both is redundant for O-LoRA and over-constrains ELLA, costing the plasticity it needs to fit each task.

Retention improves, at a cost in forward transfer. On TRACE, Muon improves retention for every method, moving BWT closer to zero (IncLoRA $-0.42 \rightarrow -0.36$, O-LoRA $-0.72 \rightarrow -0.10$, ELLA $-0.62 \rightarrow -0.07$). This gain is not free: on TRACE, Muon makes forward transfer more negative for all three methods (e.g. IncLoRA $-2.18 \rightarrow -5.72$, ELLA $-1.95 \rightarrow -3.90$), so the overall-accuracy improvements come from better task fit and retention rather than from forward transfer.

5 DISCUSSION

A natural reading of why Muon performs well on its own is that it escapes the lazy training regime. By equalizing and exploiting all singular directions, Muon may learn richer and more efficient features instead of crowding them into a few dominant ones. Under this view, forgetting is no longer driven by optimization interference but might instead reflect the genuine correlation between tasks: once features occupy distinct orthogonal directions, tasks interfere only to the extent that they are truly correlated.

This same geometry can explain why Muon does not combine well with dedicated CL methods. O-LoRA and ELLA already shrink and constrain each update to limit interference, and Muon achieves a comparable effect by construction. Stacking the two is therefore redundant for O-LoRA and over-constraining for ELLA, which loses the plasticity it needs to fit each task. Optimizer geometry and explicit CL constraints thus act as substitutes rather than complements.

6 FUTURE WORK

Additional studies are necessary to establish a more robust conclusion based on these preliminary findings. First, we will report multi-seed runs so that the gaps between methods can be assessed against seed-level noise, since our current numbers are single runs. Second, we will extend the evaluation to other benchmarks, where the behavior of Muon may differ. Third, we will extend our evaluation to larger, generative large language models (such as Llama), to verify whether the benefits of Muon’s update geometry hold at scale. Finally, we will analyze why an orthogonalized update improves retention: in particular, we want to disentangle how much of the effect comes from Muon’s normalization of the update scale versus its orthogonalization of the update direction.

REFERENCES

- Lenaic Chizat, Edouard Oyallon, and Francis Bach. On lazy training in differentiable programming. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, pp. 2937–2947, 2019.
- Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. Scaling instruction-finetuned language models. *Journal of Machine Learning Research*, 25(70):1–53, 2024.
- Shristi Das Biswas, Yue Zhang, Anwesha Pal, Radhika Bhargava, and Kaushik Roy. ELLA: Efficient lifelong learning for adapters in large language models. In Vera Demberg, Kentaro Inui, and Lluís Marquez (eds.), *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1907–1924, Rabat, Morocco, March 2026. Association for Computational Linguistics. ISBN 979-8-89176-380-7. doi: 10.18653/v1/2026.eacl-long.84. URL <https://aclanthology.org/2026.eacl-long.84/>.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://openreview.net/forum?id=pdcmv0gC9A_.
- Yebowen Hu, Timothy Ganter, Hanieh Deilamsalehy, Franck Dernoncourt, Hassan Foroosh, and Fei Liu. MeetingBank: A benchmark dataset for meeting summarization. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16409–16423, 2023.
- Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. URL <https://kellerjordan.github.io/posts/muon/>.
- Tim Tsz-Kit Lau, Qi Long, and Weijie Su. PolarGrad: A class of matrix-gradient optimizers from a unifying preconditioning perspective. *arXiv preprint arXiv:2505.21799*, 2025.
- Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, Yanru Chen, Huabin Zheng, Yibo Liu, Shaowei Liu, Bohong Yin, Weiran He, Han Zhu, Yuzhi Wang, Jianzhou Wang, Mengnan Dong, Zheng Zhang, Yongsheng Kang, Hao Zhang, Xinran Xu, Yutao Zhang, Yuxin Wu, Xinyu Zhou, and Zhilin Yang. Muon is scalable for LLM training, 2025. URL <https://arxiv.org/abs/2502.16982>.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- Pan Lu, Swaroop Mishra, Tanglin Xia, Liang Qiu, Kai-Wei Chang, Song-Chun Zhu, Oyvind Tafjord, Peter Clark, and Ashwin Kalyan. Learn to explain: Multimodal reasoning via thought chains for science question answering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.

- Swaroop Mishra, Arindam Mitra, Neeraj Varshney, Bhavdeep Sachdeva, Peter Clark, Chitta Baral, and Ashwin Kalyan. NumGLUE: A suite of fundamental yet challenging mathematical reasoning tasks. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3505–3523, 2022.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.
- Annette Rios, Nicolas Spring, Tannon Kew, Marek Kostrzewa, Andreas Säuberli, Mathias Müller, and Sarah Ebling. A new dataset and efficient baselines for document-level text simplification in german. In *Proceedings of the Third Workshop on New Frontiers in Summarization*, pp. 152–161, 2021.
- Agam Shah, Suvan Paturi, and Sudheer Chava. Trillion dollar words: A new financial dataset, task & market analysis. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 6664–6679, 2023.
- Xiao Wang, Tianze Chen, Qiming Ge, Han Xia, Rong Bao, Rui Zheng, Qi Zhang, Tao Gui, and Xuanjing Huang. Orthogonal subspace learning for language model continual learning. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 10658–10671, Singapore, December 2023a. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.715. URL <https://aclanthology.org/2023.findings-emnlp.715/>.
- Xiao Wang, Yuansen Zhang, Tianze Chen, Songyang Gao, Senjie Jin, Xianjun Yang, Zhiheng Xi, Rui Zheng, Yicheng Zou, Tao Gui, et al. TRACE: A comprehensive benchmark for continual learning in large language models. *arXiv preprint arXiv:2310.06762*, 2023b.
- Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015.
- Chenye Zhao, Yingjie Li, and Cornelia Caragea. C-STANCE: A large dataset for chinese zero-shot stance detection. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 13369–13385, 2023.

A RELATIVE FROBENIUS NORM

Relative Frobenius Norm quantifies the aggregated magnitude of parameter updates relative to their current state across all L adapter modules, calculated at training step t as:

$$\text{Global Rel-Fro} = \sqrt{\frac{\sum_{l=1}^L \|W_t^{(l)} - W_{t-1}^{(l)}\|_F^2}{\sum_{l=1}^L \|W_{t-1}^{(l)}\|_F^2}},$$

where $\|\cdot\|_F$ denotes the Frobenius norm, and $W_t^{(l)}$ represents the weight matrix of the l -th module at step t .

B EQUIVALENCE OF THE MATCHED LEARNING RATE

We justify the scaling rule $\eta_{\text{Muon}} = \eta_{\text{AdamW}} \sqrt{\max(m, n)}$ used in Section 3. The goal is to pick η_{Muon} so that a single Muon step perturbs the entries of an adapter matrix by the same root-mean-square (RMS) amount as an AdamW step taken at learning rate η_{AdamW} . Matching the per-entry update magnitude is precisely what makes the optimizer comparison fair: any remaining difference then reflects the *direction* of the update, not its scale.

For a matrix $A \in \mathbb{R}^{m \times n}$ we use the entrywise RMS

$$\text{RMS}(A) = \sqrt{\frac{1}{mn} \sum_{i,j} A_{ij}^2} = \frac{\|A\|_F}{\sqrt{mn}}. \quad (1)$$

AdamW update. The AdamW update of a weight matrix $W \in \mathbb{R}^{m \times n}$ is $\Delta W_{\text{AdamW}} = \eta_{\text{AdamW}} (\hat{m} \oslash (\sqrt{\hat{v}} + \epsilon))$, where $\oslash$ denotes entrywise division and $\hat{m}, \hat{v}$ are the bias-corrected first and second moment estimates. Under the standard idealization that each coordinate is normalized by its own running scale, i.e. $\hat{m}_{ij}/\sqrt{\hat{v}_{ij}} \approx \pm 1$, every entry of the update direction has unit magnitude, so

$$\text{RMS}(\Delta W_{\text{AdamW}}) \approx \eta_{\text{AdamW}}. \quad (2)$$

More generally $\text{RMS}(\Delta W_{\text{AdamW}}) = c \eta_{\text{AdamW}}$ for some $\mathcal{O}(1)$ constant c ; we take $c = 1$ and absorb it into the base learning rate.

Muon update. Muon replaces the momentum $G = USV^\top$ by its orthogonalization $O = \text{NewtonSchulz}(G) \approx UV^\top$ and updates $\Delta W_{\text{Muon}} = \eta_{\text{Muon}} O$. Since U and V have orthonormal columns, O has $r = \min(m, n)$ singular values all equal to one, so

$$\|O\|_F^2 = \text{tr}(O^\top O) = \text{tr}(VU^\top UV^\top) = \text{tr}(VV^\top) = r = \min(m, n). \quad (3)$$

Using $\min(m, n) \cdot \max(m, n) = mn$, the update magnitude is therefore

$$\text{RMS}(\Delta W_{\text{Muon}}) = \eta_{\text{Muon}} \frac{\|O\|_F}{\sqrt{mn}} = \eta_{\text{Muon}} \frac{\sqrt{\min(m, n)}}{\sqrt{mn}} = \frac{\eta_{\text{Muon}}}{\sqrt{\max(m, n)}}. \quad (4)$$

Matching the two. Equating the update magnitudes equation 2 and equation 4,

$$\frac{\eta_{\text{Muon}}}{\sqrt{\max(m, n)}} = \eta_{\text{AdamW}} \iff \eta_{\text{Muon}} = \eta_{\text{AdamW}} \sqrt{\max(m, n)}, \quad (5)$$

which is the rule used in the main text. $\square$

Remarks. This recovers the per-parameter update-scale rule of Liu et al. (2025), derived for large-scale pretraining and applied here per adapter matrix, so (m, n) are the dimensions of the matrix being updated. The derivation assumes zero weight decay ($\lambda = 0$) for both optimizers, so no decoupled $-\eta\lambda W$ term enters the RMS balance. We also take $c = 1$; a more accurate value should be obtained in future work from direct measurement of AdamW’s update RMS, giving $\eta_{\text{Muon}} = c \eta_{\text{AdamW}} \sqrt{\max(m, n)}$.